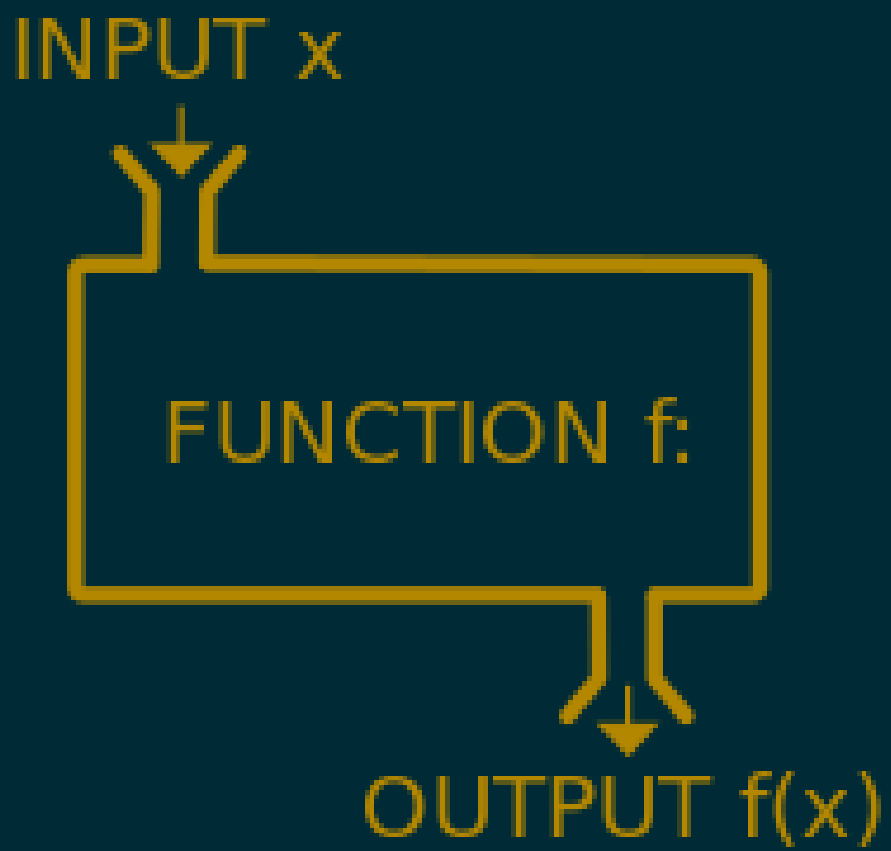
LEARN CODING

ale66

github.com/ale66/learn-coding

FUNCTIONS



Functions are a key abstraction to model nature and processes

a regular input/output or cause/effect behaviour is identified and *given a name*

```
1  The higher the temperature the quicker the veggies cook.
2
3  Cooking time is a function of the temperature in the oven.
```

FUNCTIONS IN CODING

A function is a block of code that

- has a clear input/output definition and
- executes in a separated environment

```
1  def marks2pc(marks: int):
2    ''' Convert Italian exam marks into percentages '''
3
4    pc = int((marks / 30) * 100)
5
6    return pc
```

`marks` is a *parameter* of the f.

`pc` is the *return value* of the f.

OBSERVATIONS

Functions only run when they are called (‘invoked’) within a code in execution

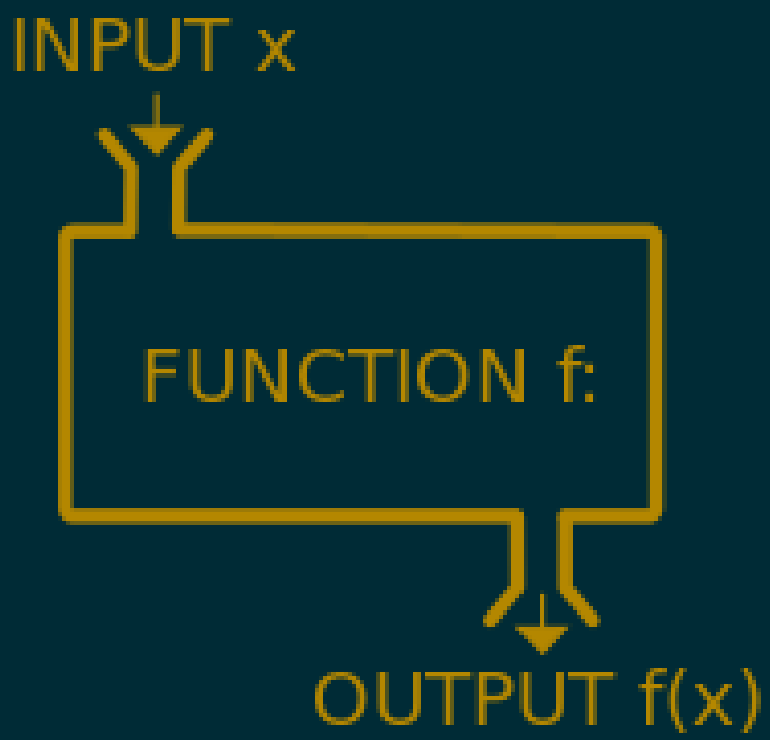
```
1  for m in my_italian_exam_marks:
2    uk_marks = marks2pc(m)
3    print(uk_marks)
```

Functions should be defined every time a block of code is required to appear more than once:

- improve readability
- improve maintenance

SYNTAX

```
1  def <name>(<parameter(s)>):
2    return <output(s)>
```



KINDS OF FUNCTIONS:

- **built-in:** come with Python, e.g., `input()`, `print()`, `float()`, `int()` etc.
- **methods:** come with types, e.g., `mylist.len()`, `mylist.append('a')`
- **external** are *imported* into the code upon demand (more later)
- **defined** we define then use them

Names of the built-in functions are reserved: avoid them as variable names

`void` FUNCTIONS

Functions may *not* return a value

they need not be to the right of an assignment symbol

```
 1  def greet(lang):
 2
 3    if lang == 'es':
 4      print('Hola!')
 5
 6    elif lang == 'fr':
 7      print('Bonjour!')
 8
 9    else:
10      print('Hello!')
11
12  # example call
13  greet('es')
```

```
Hola!
```

CALLING FUNCTIONS

PREPARING FUNCTIONS

Given a text and a character, count the number of occurrences of it

```
1  def char_counter(text, c):
2    '''returns the number of times c is found in text'''
3
4    # this cannot be empty
5    pass
```

A recent development: types

```
1  def char_counter(text: str, c: str) --> int:
2    '''returns the number of times c is found in text'''
3
4    # this cannot be empty
5    pass
```

MULTIPLE ARGUMENTS

f. can take two or more arguments (more later)

mapping calling parameters to input vars. is done by position

it is also possible to return multiple arguments

```
1  def func(primero, segundo):
2
3    # ...
4
5    return tercero, quarto
6
7  # example call
8  first_out, second_out =  func(first_in, second_in)
```

what are the bindings?

FOR REFERENCE: LIST-SPECIFIC BUILT-IN FUNCTIONS

Essentially, work with list is implemented already

```
1  l = len(my_temperatures_list)
2
3  min_val = min(my_temperatures_list)
4
5  max_val = max(my_temperatures_list)
6
7  avg_val = sum(my_temperatures_list) / l
```

FUNCTION NESTING

Example: compute summary stats for temperatures in a given month of the year.

Let's package up the list built-in functions to compute summary stats in a *tidy* code

```
1  october = [25, 26, 23, 18, 18, 19, 23, 22]
2
3  monthly_min, monthly_max, monthly_avg, days = find_tendencies(october)
```

define `find_tendencies()` by combining the built-ins.

```python
def find_tendencies(my_temperatures_list):
    '''Compute some stats on a list of temperatures'''

    l = len(my_temperatures_list)

    if l > 0:
        min_val = min(my_temperatures_list)
        max_val = max(my_temperatures_list)
        avg_val = sum(my_temperatures_list) / l

        # Return multiple values separated by commas (creates a tuple)
        return min_val, max_val, avg_val, l

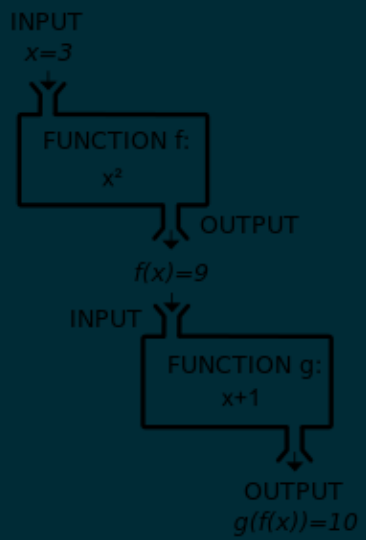
    else:
      print('Empty input data, defaulting to None')
      return None, None, None, 0
```

Notice the multiple exit points with two `return` points

Visualise it on Pythontutor

NESTED CALLS

A **return** value is **input parameter** to another f.




```
1  def square(x):
2    return x**2
3
4  def average_of_three(x, y, z):
5    return (x+y+z)/3
6
7  print(average_of_three(square(10), square(20), square(30)))
```

FUNCTIONS AND VISIBILITY

VISIBILITY OF VARIABLES

argument: what the caller sends to the function, e.g., `'es'`

parameter: the local, received value, e.g., var. `lang`

F. are executed in a *container*, they should

- only *see* their parameters,
- not see the variables of the calling code

Try to make no assumption on who's calling

F. execute in a separated environment with *private* variables

External vars. (from the caller) may be read **but not changed**

```
 1  def func(primero, segundo):
 2
 3      print(f'Inside here, primero is really first_in: {primero} and {first_in}')
 4      tercero = 3
 5      quarto = 4
 6
 7      return tercero, quarto
 8
 9  # example call
10  first_in = 1
11  second_in = 2
12
13  first_out, second_out =  func(first_in, second_in)
```

```
Inside here, primero is really first_in: 1 and 1
```

Visualise it on Pythontutor

```python
def func(primero, segundo):

    first_in = 0
    print(f'Inside here, is primero really first_in? {primero} is {first_in}')
    tercero = 3
    quarto = 4

    return tercero, quarto

first_in = 1
second_in = 2

first_out, second_out =  func(first_in, second_in)

print(f'Outside, first_in is {first_in}')
```

```
Inside here, is primero really first_in? 1 is 0
Outside, first_in is 1
```

A new `first_in` variable is available, but only inside `func` unless we specify `global` or `nonlocal` (not recommended)

QUIZZES

QUIZ 1/4

What is the default return value for a function that does not return any value explicitly?

- `None`
- `int`
- `null`
- `str`

QUIZ 2/4!

Which of the following items are present in the function header?

- function name
- function name and parameter list
- parameter list
- return value

QUIZ 3/4!

Which of the following keywords marks the beginning of the function block?

- `fun`
- `define`
- `def`
- `function`

QUIZ 4/4!

Which of the following function definition does not return any value?

- print all integers from 1 to 100.
- return a random integer from 1 to 100.
- check whether the current second is an integer from 1 to 100.
- convert an uppercase letter to lowercase.

ANSWERS:

What is the default return value ...?

- None

Which ... present in the function header?

- function name and parameter list

Which ... marks the beginning of the function block?

- `def`

Which ... does not return any value?

- print all integers from 1 to 100